

```
False
ghci> 3 > 2
True
```

Its type class definition is as follows:

```
class (Eq a) => Ord a where
  compare      :: a -> a -> Ordering
  (<), (<=), (>), (>=) :: a -> a -> Bool
  max, min     :: a -> a -> a

  compare x y = if x == y then EQ
                else if x <= y then LT
                else GT

  x < y = case compare x y of { LT -> True; _ -> False }
  x <= y = case compare x y of { GT -> False; _ -> True }
  x > y = case compare x y of { GT -> True; _ -> False }
  x >= y = case compare x y of { LT -> False; _ -> True }

  max x y = if x <= y then y else x
  min x y = if x <= y then x else y
```

The *Ord* type class needs to be a sub-class of the *Eq* class because we should be able to test for equality of two values if they need to be compared. This is also defined as a constraint in the class definition. Seven functions are provided and a minimal definition given in the code snippet. The instance definitions for *Char* and *Int* types are available from *libraries/ghc-prim/GHC/Classes.hs* in the ghc source code.

The *Enum* type class is for types whose values can be listed in an order for which you can find predecessor and successor elements. For example:

```
ghci> succ 'a'
b
ghci> pred EQ
LT
```

The class definition for *Enum* is given below:

```
class Enum a where
  succ      :: a -> a
  pred      :: a -> a
  toEnum    :: Int -> a
  fromEnum  :: a -> Int

  enumFrom      :: a -> [a]
  enumFromThen  :: a -> a -> [a]
  enumFromTo    :: a -> a -> [a]
  enumFromThenTo :: a -> a -> a -> [a]

  succ = toEnum . (+ 1) . fromEnum
  pred = toEnum . (subtract 1) . fromEnum
```

```
enumFrom x      = map toEnum [fromEnum x ..]
enumFromThen x y = map toEnum [fromEnum x, fromEnum
y ..]
enumFromTo x y   = map toEnum [fromEnum x ..
fromEnum y]
enumFromThenTo x1 x2 y = map toEnum [fromEnum x1, fromEnum
x2 .. fromEnum y]
```

The instance for type ordering for the *Enum* class is as follows:

```
instance Enum Ordering where
  succ LT = EQ
  succ EQ = GT
  succ GT = error "Prelude.Enum.Ordering.succ: bad argument"

  pred GT = EQ
  pred EQ = LT
  pred LT = error "Prelude.Enum.Ordering.pred: bad argument"

  toEnum n | n == 0 = LT
           | n == 1 = EQ
           | n == 2 = GT
  toEnum _ = error "Prelude.Enum.Ordering.toEnum: bad
argument"

  fromEnum LT = 0
  fromEnum EQ = 1
  fromEnum GT = 2

  -- Use defaults for the rest
  enumFrom      = boundedEnumFrom
  enumFromThen  = boundedEnumFromThen
```

You can find the definition and instance definitions for *Char* and *Ordering* in *libraries/base/GHC/Enum.lhs* in the ghc source code.

The *Show* type class lists a *show* function to display or print data. For example:

```
ghci> show 3.1415
"3.1415"
ghci> show True
"True"
```

The above code works for both 'Float' and 'Bool' because there are instance definitions for each for the *Show* type class.

The read function for the 'Read' type class takes as input a 'String' and converts it to an appropriate data type, if possible. For example:

```
ghci> read "1" + 2.0
3.0
ghci> read "False" || True
True
```